

Министерство науки и высшего образования Российской
Федерации

Кафедра информационных технологий

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту

«Конвертер изображений в ASCII-арт»

Выполнил: Насекин И.Д.

Группа: 253-321

Дисциплина: Основы программирования

Содержание

1	Введение	2
2	Анализ предметной области	2
2.1	Что такое ASCII-арт	2
2.2	Обоснование выбора технологий	2
3	Техническое задание	3
3.1	Функциональные требования	3
3.2	Нефункциональные требования	3
4	Архитектура системы	3
5	Проектирование базы данных	4
5.1	Описание предметной области	4
5.2	ER-диаграмма	4
5.3	Логическая схема	4
5.4	DDL — создание таблиц	5
5.5	SQL-запросы	6
6	Реализация	6
6.1	Алгоритм конвертации	6
6.2	Многопоточность	7
6.3	Интеграция с БД	7
7	Руководство пользователя	7
7.1	Установка	7
7.2	Работа с программой	7
8	Заключение	8

1 Введение

Представленная работа посвящена разработке десктопного приложения «*Конвертер изображений в ASCII-арт*» на языке Python. Программа преобразует растровые изображения в текстовое представление, состоящее из символов ASCII, используя яркость каждого пикселя для подбора подходящего символа из заданного набора.

Цель работы — разработать полнофункциональное GUI-приложение, удовлетворяющее требованиям технического задания, а также спроектировать и реализовать базу данных для хранения истории конвертаций.

Задачи:

1. Реализовать алгоритм конвертации изображений в ASCII-арт.
2. Разработать графический интерфейс на основе tkinter.
3. Обеспечить поддержку GIF-анимации и анимации вращения.
4. Спроектировать реляционную базу данных (предметная область, ER-диаграмма, логическая схема, SQL-запросы).
5. Интегрировать базу данных в приложение.

2 Анализ предметной области

2.1 Что такое ASCII-арт

ASCII-арт — способ визуального представления изображений с помощью символов таблицы ASCII. Каждому пикселю исходного изображения ставится в соответствие символ, яркость которого максимально близка к яркости пикселя. Тёмным пикселям соответствуют «тяжёлые» символы (@#\$), светлым — лёгкие (.). Ключевой особенностью является то, что символ текстового шрифта выше, чем шире, поэтому высота результата умножается на коэффициент 0,5 для сохранения пропорций.

2.2 Обоснование выбора технологий

- **Python 3.10+** — лаконичный синтаксис, богатая стандартная библиотека, поддержка аннотаций типов.
- **Pillow (PIL)** — де-факто стандарт обработки изображений в Python; поддерживает PNG, JPEG, BMP, GIF.
- **tkinter** — входит в стандартную поставку Python, не требует установки сторонних зависимостей.
- **SQLite (модуль sqlite3)** — встроенная реляционная СУБД, не требует сервера, файл базы данных переносится вместе с проектом.

3 Техническое задание

3.1 Функциональные требования

1. Поддержка форматов: PNG, JPEG, BMP, GIF.
2. Три набора символов: *standard*, *extended*, *gradient*.
3. Настройка ширины и высоты результата (или автоматическое вычисление высоты с сохранением пропорций).
4. Инверсия яркости.
5. Цветной вывод (ANSI-коды 24-bit).
6. Анимация GIF-изображений в реальном времени.
7. Анимация вращения изображения на 360°.
8. Пакетная обработка нескольких файлов с сохранением в папку.
9. Сохранение результата в файл `.txt`.
10. Автосохранение настроек между сеансами.
11. Ведение истории конвертаций в базе данных SQLite.

3.2 Нефункциональные требования

- Конвертация выполняется в отдельном потоке (не блокирует UI).
- Минимальный размер окна: 800×500 пикселей.
- Автодебаунс: повторная конвертация запускается не чаще чем раз в 400 мс.

4 Архитектура системы

Приложение состоит из пяти модулей:

`main.py` Точка входа. Создает экземпляр `App` и запускает главный цикл событий.

`gui.py` Графический интерфейс. Содержит класс `App`, унаследованный от `tk.Tk`. Обрабатывает все пользовательские события, запускает конвертацию в фоновом потоке.

`converter.py` Алгоритм конвертации. Функции `convert_image()`, `convert_gif_frames()`, `generate_rotation_frames()`.

`settings.py` Загрузка/сохранение настроек в `settings.json`.

`database.py` Работа с SQLite: инициализация схемы, добавление записей, пять именованных запросов.

5 Проектирование базы данных

5.1 Описание предметной области

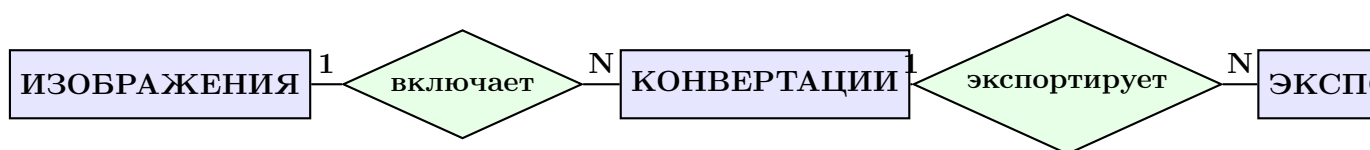
В базе данных хранится история работы пользователя с приложением:

- **Изображение** — исходный файл, зарегистрированный при первом открытии. Хранит метаданные: имя, путь, размер в байтах, пиксельные размеры.
- **Конвертация** — один акт обработки изображения с заданными параметрами. Фиксирует ширину/высоту ASCII-результата, набор символов, флаги инверсии и цвета, итоговый текст и временную метку.
- **Экспорт** — сохранение результата конвертации в файл. Хранит путь назначения, формат и дату.

Одно изображение может конвертироваться многократно (разные параметры). Один результат конвертации может экспортироваться в несколько файлов.

5.2 ER-диаграмма

ER-диаграмма в нотации Чена представлена в файле `er_diagram.tex`. Ниже приведена её упрощённая версия:



5.3 Логическая схема

Логическая диаграмма (реляционная модель) описана в файле `logical_diagram.tex`. В реляционном представлении:

Таблица	Столбцы
images	<u>id</u> , filename, filepath (UNIQUE), file_size, width_px, height_px, added_at
conversions	<u>id</u> , image_id →images, ascii_width, ascii_height, charset, inverted, colored, result_text, converted_at
exports	<u>id</u> , conversion_id →conversions, export_path, export_format, exported_at

5.4 DDL — создание таблиц

Листинг 1: Скрипт создания таблиц

```

1 CREATE TABLE IF NOT EXISTS images (
2     id                INTEGER PRIMARY KEY AUTOINCREMENT,
3     filename          TEXT      NOT NULL,
4     filepath          TEXT      NOT NULL UNIQUE,
5     file_size         INTEGER,
6     width_px          INTEGER,
7     height_px         INTEGER,
8     added_at          DATETIME DEFAULT CURRENT_TIMESTAMP
9 );
10
11 CREATE TABLE IF NOT EXISTS conversions (
12     id                INTEGER PRIMARY KEY AUTOINCREMENT,
13     image_id          INTEGER NOT NULL REFERENCES images(id) ON DELETE
14         CASCADE,
15     ascii_width        INTEGER NOT NULL,
16     ascii_height       INTEGER,
17     charset            TEXT      NOT NULL DEFAULT 'standard',
18     inverted           INTEGER NOT NULL DEFAULT 0,
19     colored            INTEGER NOT NULL DEFAULT 0,
20     result_text        TEXT      DEFAULT '',
21     converted_at       DATETIME DEFAULT CURRENT_TIMESTAMP
22 );
23
24 CREATE TABLE IF NOT EXISTS exports (
25     id                INTEGER PRIMARY KEY AUTOINCREMENT,
26     conversion_id      INTEGER NOT NULL
27         REFERENCES conversions(id) ON DELETE CASCADE,
28     export_path         TEXT      NOT NULL,
29     export_format       TEXT      NOT NULL DEFAULT 'txt',
30     exported_at         DATETIME DEFAULT CURRENT_TIMESTAMP
31 );

```

5.5 SQL-запросы

В файле `queries.sql` и модуле `database.py` реализовано пять именованных запросов.

Запрос 1 — все конвертации конкретного изображения:

```
1 SELECT c.id, c.ascii_width, c.charset, c.converted_at
2 FROM   conversions c
3 WHERE  c.image_id = :image_id
4 ORDER  BY c.converted_at DESC;
```

Запрос 2 — история последних 20 конвертаций (JOIN):

```
1 SELECT i.filename, c.ascii_width, c.charset, c.converted_at
2 FROM   conversions c
3 JOIN   images i ON c.image_id = i.id
4 ORDER  BY c.converted_at DESC LIMIT 20;
```

Запрос 3 — статистика по наборам символов:

```
1 SELECT charset, COUNT(*) AS total
2 FROM   conversions
3 GROUP  BY charset ORDER BY total DESC;
```

Запрос 4 — экспорты с деталями (тройной JOIN):

```
1 SELECT e.id, i.filename, c.charset, e.export_path, e.exported_at
2 FROM   exports e
3 JOIN   conversions c ON e.conversion_id = c.id
4 JOIN   images      i ON c.image_id      = i.id
5 ORDER  BY e.exported_at DESC;
```

Запрос 5 — топ изображений по числу конвертаций:

```
1 SELECT i.filename, COUNT(c.id) AS conv_count
2 FROM   images i
3 LEFT JOIN conversions c ON i.id = c.image_id
4 GROUP  BY i.id ORDER BY conv_count DESC LIMIT 5;
```

6 Реализация

6.1 Алгоритм конвертации

Основная функция `_convert_frame()` принимает изображение PIL, масштабирует его до заданного размера `width × height` и для каждого пикселя вычисляет соответствующий символ:

$$\text{char} = \text{charset} \left[\left\lfloor \frac{b}{255} \cdot (n - 1) \right\rfloor \right] \quad (1)$$

где b — яркость пикселя (0–255), n — длина набора символов.

При инверсии яркость заменяется на $b' = 255 - b$.

При цветном режиме каждый символ оборачивается в ANSI-последовательность трёхкомпонентного цвета: `\033[38;2;R;G;Bm`.

6.2 Многопоточность

Конвертация выполняется в фоновом потоке (`threading.Thread`). Результат передаётся в главный поток через `self.after(0, callback)`, что исключает блокировку UI. Счётчик поколений (`_convert_gen`) отменяет устаревшие результаты при быстром изменении параметров.

6.3 Интеграция с БД

При каждой успешной конвертации:

1. Вызывается `db.add_image()` — регистрирует файл (IGNORE при повторном открытии).
2. Вызывается `db.add_conversion()` — сохраняет параметры и результат.

При сохранении файла вызывается `db.add_export()`. Кнопка «История» открывает диалог с результатами `q2_recent_history()`.

7 Руководство пользователя

7.1 Установка

1. Установить Python 3.10+.
2. Выполнить `pip install Pillow` или `uv sync`.
3. Запустить: `python main.py`.

7.2 Работа с программой

1. Нажать «Открыть...» и выбрать изображение.
2. Настроить ширину символов в поле *Ширина*.
3. Выбрать набор символов (Standard / Extended / Gradient).
4. При необходимости включить *Инверсию* или *Цвет*.

5. Нажать «**Конвертировать**» или подождать авто-конвертации.
6. Сохранить результат: «**Сохранить в .txt...**».
7. Просмотреть историю: «**История**».

8 Заключение

В ходе выполнения курсового проекта разработано десктопное приложение «Конвертер изображений в ASCII-арт», соответствующее всем пунктам технического задания. Реализованы: алгоритм пиксельной конвертации, GUI на tkinter, поддержка GIF-анимации и вращения, пакетная обработка.

Спроектирована и реализована реляционная база данных SQLite из трёх таблиц (`images`, `conversions`, `exports`) с пятью именованными запросами. Составлены ER-диаграмма в нотации Чена и логическая схема БД.

Возможные направления развития проекта: экспорт в HTML с сохранением цвета, поддержка видеофайлов через ffmpeg, веб-интерфейс на Flask.